

Práctica 2: Analizador sintáctico de descenso recursivo

Compiladores 2026-2

Profesora: Ariel Adara Mercado Martínez

Alumno

Edwin Rodríguez Nevarez - 312227441

Objetivo

Que el alumno conozca e implemente los principios del análisis sintáctico descendente recursivo, aplicando técnicas de transformación de gramáticas —eliminación de ambigüedad, eliminación de recursividad izquierda y factorización izquierda— para construir un analizador sintáctico predictivo que, junto con el analizador léxico desarrollado con Flex, permita reconocer programas válidos del lenguaje C_1.

Desarrollo

1. Determinar los conjuntos N , Σ y el símbolo inicial S .

A partir de las reglas de producción de la gramática $G = (N, \Sigma, P, S)$, definimos los siguientes conjuntos:

- **No-Terminales (N):** Son los símbolos que pueden ser reemplazados por otras secuencias de símbolos.

$$N = \{\text{programa, declaraciones, declaracion, tipo, lista_var, sentencias, sentencia, expresion}\}$$

- **Terminales (Σ):** Son los tokens que ya no se pueden derivar.

$$\Sigma = \{\text{int, float, if, else, while, identificador, numero, ;, ,, =, +, -, *, /, (,)}\}$$

- **Símbolo inicial (S):** Es el No-Terminal raíz desde el cual se comienza a derivar cualquier cadena válida.

$$S = \text{programa}$$

2. Mostrar el proceso de eliminación de ambigüedad o justificar, en caso de no ser necesario.

La gramática original es ambigua en *expresion*, porque no tiene precedencia ni asociatividad para los operadores. Esto podría hacer mas de un árbol de derivación válido para una misma cadena.

Para eliminar la ambigüedad, se dividieron las operaciones con nuevos No-Terminales (*termino* y *factor*) para poner la jerarquía matemática (multiplicación/división antes que suma/resta):

- $expresion \rightarrow expresion + termino \mid expresion - termino \mid termino$

- $termino \rightarrow termino * factor \mid termino / factor \mid factor$
- $factor \rightarrow (expresion) \mid identificador \mid numero$

3. **Mostrar el proceso de eliminación de la recursividad izquierda o justificar, en caso de no ser necesario.**

Tenemos recursividad por la izquierda en las producciones de *declaraciones*, *lista_var*, *sentencias*, *expresion* y *termino*. Para que el analizador descendente no caiga en un ciclo infinito, usamos la transformación $A \rightarrow A\alpha \mid \beta \implies A \rightarrow \beta A', A' \rightarrow \alpha A' \mid \epsilon$:

Para declaraciones:

- $declaraciones \rightarrow declaracion declaraciones'$
- $declaraciones' \rightarrow declaracion declaraciones' \mid \epsilon$

Para lista_var:

- $lista_var \rightarrow identificador lista_var'$
- $lista_var' \rightarrow , identificador lista_var' \mid \epsilon$

Para sentencias:

- $sentencias \rightarrow sentencia sentencias'$
- $sentencias' \rightarrow sentencia sentencias' \mid \epsilon$

Para expresion:

- $expresion \rightarrow termino expresion'$
- $expresion' \rightarrow + termino expresion' \mid - termino expresion' \mid \epsilon$

Para termino:

- $termino \rightarrow factor termino'$
- $termino' \rightarrow * factor termino' \mid / factor termino' \mid \epsilon$

4. **Mostrar el proceso de factorización izquierda o justificar, en caso de no ser necesario.**

No es necesario aplicar factorización izquierda. Después de eliminar la ambigüedad y la recursividad izquierda, no tenemos ninguna regla que comparta un prefijo común. Por ejemplo, las producciones de *tipo* derivan en **int** o **float**, y las de *sentencia* en **identificador**, **if** o **while**, por lo tanto no hay necesidad de factorizarlas.

5. **Nuevos conjuntos N y P .**

Nuevo conjunto de No-Terminales (N):

$$N = \{\text{programa, declaraciones, declaraciones', declaracion, tipo, lista_var, lista_var', sentencias, sentencias', sentencia, expresion, expresion', termino, termino', factor}\}$$

Nuevo conjunto de Producciones (P):

- $programa \rightarrow declaraciones\ sentencias$
- $declaraciones \rightarrow declaracion\ declaraciones'$
- $declaraciones' \rightarrow declaracion\ declaraciones' \mid \epsilon$
- $declaracion \rightarrow tipo\ lista_var\ ;$
- $tipo \rightarrow \mathbf{int} \mid \mathbf{float}$
- $lista_var \rightarrow \mathbf{identificador}\ lista_var'$
- $lista_var' \rightarrow ,\ \mathbf{identificador}\ lista_var' \mid \epsilon$
- $sentencias \rightarrow sentencia\ sentencias'$
- $sentencias' \rightarrow sentencia\ sentencias' \mid \epsilon$
- $sentencia \rightarrow \mathbf{identificador} = expresion\ ; \mid \mathbf{if}\ (expresion)sentencias\ \mathbf{else}\ sentencias \mid \mathbf{while}\ (expresion)sentencias$
- $expresion \rightarrow termino\ expresion'$
- $expresion' \rightarrow +\ termino\ expresion' \mid -\ termino\ expresion' \mid \epsilon$
- $termino \rightarrow factor\ termino'$
- $termino' \rightarrow *\ factor\ termino' \mid /\ factor\ termino' \mid \epsilon$
- $factor \rightarrow (expresion) \mid \mathbf{identificador} \mid \mathbf{numero}$

6. Definir el Analizador Léxico (*lexer.ll*) con las acciones léxicas correspondientes.

En lugar de imprimir cadenas en la salida estándar, el escáner ahora retorna los identificadores de cada token (definidos en `Symbols.hpp`) para que el Parser LL(1) pueda leerlos. Se tiene un conteo de líneas para ubicar posibles errores léxicos o sintácticos:

```
1  %{
2  #include <iostream>
3  #include <string>
4  using namespace std;
5
6  #include "headers/Symbols.hpp"
7  #include "headers/Lexer.hpp"
8
9  int line=1;
10 %}
11
12 %option c++
13 %option outfile="Lexer.cpp"
14 %option yyclass="Lexer"
15
16 /* Definicion de macros */
17 ID      [a-zA-Z_][a-zA-Z0-9_]*
18 NUMERO  [0-9]+(\.[0-9]+)?([eE][+-]?[0-9]+)?
19
20 %%
```

```

21
22 [ \t\r]+ { /* Ignoramos espacios */ }
23 "\n"     { line++; }
24
25 "int"     { return INT; }
26 "float"   { return FLOAT; }
27 "if"      { return IF; }
28 "else"    { return ELSE; }
29 "while"   { return WHILE; }
30
31 ";"       { return PYC; }
32 ","       { return COMA; }
33 "="       { return ASIG; }
34 "+"       { return MAS; }
35 "-"       { return MENOS; }
36 "*"       { return MUL; }
37 "/"       { return DIV; }
38 "("       { return LPAR; }
39 ")"       { return RPAR; }
40
41 {ID}      { return IDENTIFICADOR; }
42 {NUMERO}  { return NUMERO; }
43
44 .         { cout << "ERROR LEXICO en linea " << line << ": " << yytext
45           << endl; }
46 %%
47
48 int yyFlexLexer::yywrap(){
49     return 1;
50 }
51
52 int Lexer::getLine()
53 {
54     return line;
55 }

```

7. Implementar el Analizador Sintáctico (*Parser.cpp*) de descenso recursivo.

Se implementó un Parser Predictivo Descendente LL(1). Para ello, se definió una enumeración completa de tokens y No-Terminales en el archivo `Symbols.hpp`, se declararon las funciones en `Parser.hpp` y se programó la lógica de las producciones en `Parser.cpp`.

Se agregó la función auxiliar `match(Token expected)`, cuya labor principal es comparar el token actual provisto por el analizador léxico contra el token que la gramática espera en ese punto específico. Si coinciden, consume el token avanzando al siguiente; si no, lanza un mensaje de error detallando la línea y detiene la ejecución mediante `exit(1)` para evitar fallas en cascada y ciclos infinitos.

A continuación se presentan los códigos completos de las cabeceras y la implementación del analizador sintáctico:

Archivo headers/Symbols.hpp:

```

1 #ifndef __SYMBOLS_HPP__

```

```

2 #define __SYMBOLS_HPP__
3
4 /** * Gramatica G(N, T, P, S)
5  * S = programa
6  * P = {
7  * programa -> declaraciones sentencias
8  * declaraciones -> declaracion declaraciones_prima
9  * declaraciones_prima -> declaracion declaraciones_prima | epsilon
10 * declaracion -> tipo lista_var ;
11 * tipo -> int | float
12 * lista_var -> identificador lista_var_prima
13 * lista_var_prima -> , identificador lista_var_prima | epsilon
14 * sentencias -> sentencia sentencias_prima
15 * sentencias_prima -> sentencia sentencias_prima | epsilon
16 * sentencia -> identificador = expresion ; | if ( expresion )
    sentencias else sentencias | while ( expresion ) sentencias
17 * expresion -> termino expresion_prima
18 * expresion_prima -> + termino expresion_prima | - termino
    expresion_prima | epsilon
19 * termino -> factor termino_prima
20 * termino_prima -> * factor termino_prima | / factor termino_prima |
    epsilon
21 * factor -> ( expresion ) | identificador | numero
22 * }
23 */
24
25 enum Token {
26     t_eof = 0, primero,
27     INT, FLOAT, IF, ELSE, WHILE,
28     PYC, COMA, ASIG,
29     MAS, MENOS, MUL, DIV,
30     LPAR, RPAR,
31     IDENTIFICADOR, NUMERO
32 };
33
34 enum NoTerm {
35     strEmpty,
36     nt_programa, nt_declaraciones, nt_declaraciones_prima,
37     nt_declaracion, nt_tipo, nt_lista_var, nt_lista_var_prima,
38     nt_sentencias, nt_sentencias_prima, nt_sentencia,
39     nt_expresion, nt_expresion_prima,
40     nt_termino, nt_termino_prima, nt_factor
41 };
42
43 #endif // !__SYMBOLS_HPP__

```

Archivo headers/Parser.hpp:

```

1 #ifndef PARSER_H
2 #define PARSER_H
3 #include <string>
4 #include "Lexer.hpp"
5 #include "Symbols.hpp"
6 using namespace std;
7

```

```

8 namespace C_1
9 {
10     class Parser
11     {
12     private:
13         Lexer *lexer;
14         int token;
15
16     public:
17         Parser();
18         Parser(Lexer *lexer);
19         ~Parser();
20
21         void programa(); //funcion del simbolo inicial
22         void declaraciones();
23         void declaraciones_prima();
24         void declaracion();
25         void tipo();
26         void lista_var();
27         void lista_var_prima();
28         void sentencias();
29         void sentencias_prima();
30         void sentencia();
31         void expresion();
32         void expresion_prima();
33         void termino();
34         void termino_prima();
35         void factor();
36
37         Token eat();
38         void error(string msg);
39         void parse();
40         void match(Token expected); // Funcion para avanzar tokens
41     };
42 }
43 #endif

```

Archivo Parser.cpp:

```

1 #include "headers/Parser.hpp"
2 #include <iostream>
3 #include <cstdlib> // Para usar exit()
4
5 namespace C_1
6 {
7     Parser::Parser(/* args */) {}
8
9     Parser::Parser(Lexer *lexer) {
10         this->lexer = lexer;
11     }
12
13     Parser::~Parser() {}
14
15     void Parser::programa(){
16         declaraciones();

```

```

17     sentencias();
18 }
19
20 void Parser::declaraciones(){
21     declaracion();
22     declaraciones_prima();
23 }
24
25 void Parser::declaraciones_prima(){
26     if (token == INT || token == FLOAT) {
27         declaracion();
28         declaraciones_prima();
29     }
30 }
31
32 void Parser::declaracion(){
33     tipo();
34     lista_var();
35     match(PYC);
36 }
37
38 void Parser::tipo(){
39     if (token == INT) {
40         match(INT);
41     } else if (token == FLOAT) {
42         match(FLOAT);
43     } else {
44         error("Se esperaba un tipo de dato (int o float)");
45     }
46 }
47
48 void Parser::lista_var(){
49     match(IDENTIFICADOR);
50     lista_var_prima();
51 }
52
53 void Parser::lista_var_prima(){
54     if (token == COMA) {
55         match(COMA);
56         match(IDENTIFICADOR);
57         lista_var_prima();
58     }
59 }
60
61 void Parser::sentencias(){
62     sentencia();
63     sentencias_prima();
64 }
65
66 void Parser::sentencias_prima(){
67     if (token == IDENTIFICADOR || token == IF || token == WHILE) {
68         sentencia();
69         sentencias_prima();
70     }

```

```

71 }
72
73 void Parser::sentencia(){
74     if (token == IDENTIFICADOR) {
75         match(IDENTIFICADOR);
76         match(ASIG);
77         expresion();
78         match(PYC);
79     } else if (token == IF) {
80         match(IF);
81         match(LPAR);
82         expresion();
83         match(RPAR);
84         sentencias();
85         match(ELSE);
86         sentencias();
87     } else if (token == WHILE) {
88         match(WHILE);
89         match(LPAR);
90         expresion();
91         match(RPAR);
92         sentencias();
93     } else {
94         error("Se esperaba una sentencia valida");
95     }
96 }
97
98 void Parser::expresion(){
99     termino();
100     expresion_prima();
101 }
102
103 void Parser::expresion_prima(){
104     if (token == MAS) {
105         match(MAS);
106         termino();
107         expresion_prima();
108     } else if (token == MENOS) {
109         match(MENOS);
110         termino();
111         expresion_prima();
112     }
113 }
114
115 void Parser::termino(){
116     factor();
117     termino_prima();
118 }
119
120 void Parser::termino_prima(){
121     if (token == MUL) {
122         match(MUL);
123         factor();
124         termino_prima();

```



```

125     } else if (token == DIV) {
126         match(DIV);
127         factor();
128         termino_prima();
129     }
130 }
131
132 void Parser::factor(){
133     if (token == LPAR) {
134         match(LPAR);
135         expresion();
136         match(RPAR);
137     } else if (token == IDENTIFICADOR) {
138         match(IDENTIFICADOR);
139     } else if (token == NUMERO) {
140         match(NUMERO);
141     } else {
142         error("Se esperaba un factor (numero, identificador o '(')");
143     }
144 }
145
146 Token Parser::eat(){
147     return (Token) lexer->yylex();
148 }
149
150 void Parser::error(string msg){
151     cout << "ERROR DE SINTAXIS: " << msg << " en la linea " << lexer->
    getLine() << endl;
152     exit(1); // Detenemos el parser al primer error para evitar ciclos
    infinitos
153 }
154
155 void Parser::match(Token expected){
156     if (token == expected) {
157         token = eat();
158     } else {
159         error("Se encontro un token inesperado");
160     }
161 }
162
163 void Parser::parse(){
164     token = eat();
165     programa();
166
167     if(token == t_eof)
168         cout << "La cadena es aceptada" << endl;
169     else
170         cout << "La cadena no pertenece al lenguaje generado por la
    gramatica" << endl;
171 }
172 }

```

```
• edwin-rodriguez@edwin-rodriguez-OMEN-by-HP-Laptop-15-dclxxx:~/Escritorio/ciencias/semestre 8/Compiladores/Laboratorio/Prácticas/P2/src$ ./analyzer prueba
La cadena es aceptada
```

Figura 1: Ejecución exitosa del analizador sintáctico aceptando el archivo de prueba.

Extras

8. Documentar el código.

Se añadió documentación con comentarios estándar en los archivos, describiendo el propósito de la clase, el funcionamiento de la función auxiliar `match()` y en cada método con las reglas de producción.

9. Proponer 4 archivos de prueba nuevos, 2 válidos y 2 inválidos.

Como la gramática dice que todo programa debe tener una sección de declaraciones y una sección de sentencias ($\text{programa} \rightarrow \text{declaraciones sentencias}$):

Archivos Válidos:

- `prueba_valida_1`: Una declaración simple y asignaciones matemáticas.

```
1 int base, altura;
2 float area;
3 base = 10;
4 altura = 20;
5 area = (base * altura) / 2;
```

- `prueba_valida_2`: Declaraciones múltiples y uso de sentencias de control anidadas.

```
1 float x, y;
2 int iterador;
3 x = 1.5;
4 while (iterador)
5     if (x)
6         iterador = iterador - 1;
7     else
8         x = x + 1.0;
```

Archivos Inválidos:

- `prueba_invalida_1`: Error de sintaxis por falta de un símbolo terminal obligatorio (falta el punto y coma en la declaración).

```
1 int a
2 a = 5;
```

- `prueba_invalida_2`: Error de la estructura de la gramática al intentar realizar una declaración después de haber iniciado la sección de sentencias.

```
1 int a;
2 a = 5;
3 float b;
```

10. **Modificar el Makefile para poder simplificar el proceso de compilación de la práctica.**

Se creó un archivo Makefile para automatizar la generación del escáner con Flex y la compilación de los tres archivos de C++ (Lexer.cpp, Parser.cpp y main.cpp) para un solo ejecutable llamado **analyzer**:

```
1 # Variables de compilacion
2 CXX = g++
3 FLEX = flex++
4 TARGET = analyzer
5
6 # Regla principal
7 all: $(TARGET)
8
9 # Vinculacion y generacion del ejecutable final
10 $(TARGET): Lexer.cpp Parser.cpp main.cpp
11     $(CXX) Lexer.cpp Parser.cpp main.cpp -o $(TARGET)
12
13 # Generacion del codigo C++ del analizador lexico
14 Lexer.cpp: lexer.ll
15     $(FLEX) lexer.ll
16
17 # Limpieza de archivos autogenerados y binarios
18 clean:
19     rm -f $(TARGET) Lexer.cpp
```